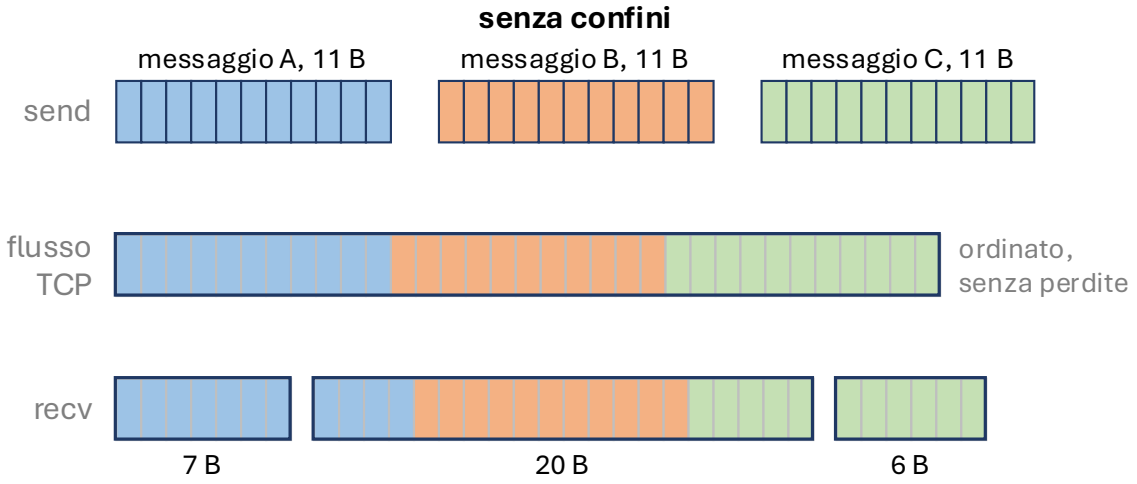
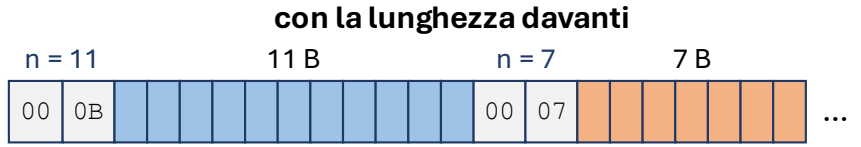
Lab 1: TCP e MQTT

https://github.com/MarcoEsposito890/iomt-labs-students

Teoria: TCP è un flusso di byte

TCP garantisce che i byte arrivino tutti e nell'ordine giusto, ma non dove finisce un messaggio: una recv restituisce i byte arrivati fino a quel momento. Dove finisce un messaggio lo deve scrivere il programma.





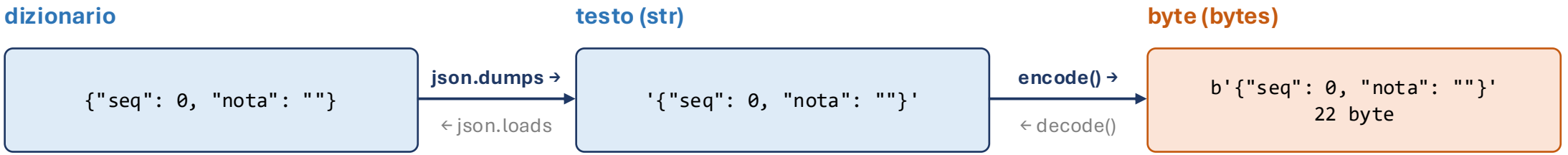
1. leggi 2 B: la lunghezza n (big-endian, ≤ 65 535)
2. leggi esattamente n B: un messaggio intero
3. ricomincia: le recv possono tagliare ovunque

La soluzione più semplice: scrivere davanti a ogni messaggio quanti byte è lungo. MQTT fa così (la Remaining Length della Parte 3). L'altra strada, un carattere separatore, funziona solo se quel carattere non compare mai nel messaggio.

Teoria: Cloud e MQTT, «Scope, transport and versions», «Remaining Length encoding».

Python: testo, byte e JSON

Sulla rete viaggiano solo **byte**. Un testo (str) diventa byte con **encode**, e torna testo con decode; un dizionario diventa testo con **json.dumps**, e torna dizionario con json.loads.




```
import json
d = {"seq": 0, "nota": ""}        # un dizionario
testo = json.dumps(d)                # '{"seq": 0, "nota": ""}'   (str)
dati = testo.encode()                # b'{"seq": 0, "nota": ""}'  (bytes): si può mandare
len(dati)                            # 22: quanti byte
json.loads(dati.decode())            # di nuovo il dizionario
```

Dizionario → json.dumps → testo → encode → byte: è la strada di ogni messaggio JSON. Un b davanti alle virgolette, b"...", indica byte, non testo.

Documentazione di Python: tipi str e bytes, modulo json.

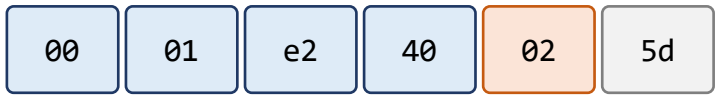
Python: struct, numeri in byte

Un numero, per viaggiare, va scritto in un numero fisso di byte. **struct.pack** lo fa secondo un formato deciso da noi; **struct.unpack** fa il contrario, e restituisce sempre una tupla.

lettera	che cosa scrive	byte
`!`	ordine dei byte: big-endian, «di rete»	—
`H`	intero senza segno, 0 … 65 535	2 B
`I`	intero senza segno, 0 … 4 294 967 295	4 B
`B`	intero senza segno, 0 … 255	1 B
`f`	numero con la virgola (float32)	4 B

esempio: una lunghezza e una decisione

```
import struct
struct.pack("!H", 22)          # b'\x00\x16': 22 in 2 byte
struct.unpack("!H", b"\x00\x16")   # (22,): una tupla
struct.pack("!IBB", 123456, 2, 93)   # 6 byte
```



tempo in ms: 4 byte — classe — 93 %

Il «!» davanti al formato fissa l'ordine dei byte (big-endian, quello delle reti): chi legge deve usare lo stesso formato, altrimenti ottiene un altro numero.

Documentazione di Python, modulo struct (format characters, byte order).

Socket TCP: aprire la connessione

Una connessione TCP ha due parti: chi **aspetta** su una porta (il server, qui il ricevitore) e chi **si collega** a quella porta (il client, qui il mittente). Prima di scambiare byte, la connessione va aperta in quattro passi.

ricevitore (server)

`1 · create_server(("127.0.0.1", 0))`

apre una porta e ci si mette in ascolto; 0: il numero lo sceglie il sistema

`2 · server.getsockname()[1]`

legge quale porta è stata scelta, per dirla al mittente

mittente (client)

`3 · create_connection(("127.0.0.1", porta))`

il mittente si collega a quella porta

`4 · conn, _ = server.accept()`

aspetta il mittente; quando arriva, restituisce conn, il socket per parlargli

```
import socket

# ricevitore
server = socket.create_server(
    ("127.0.0.1", 0))          # 1
porta = server.getsockname()[1] # 2
conn, _ = server.accept()       # 4

# mittente (un altro thread)
mittente = socket.create_connection(
    ("127.0.0.1", porta))      # 3

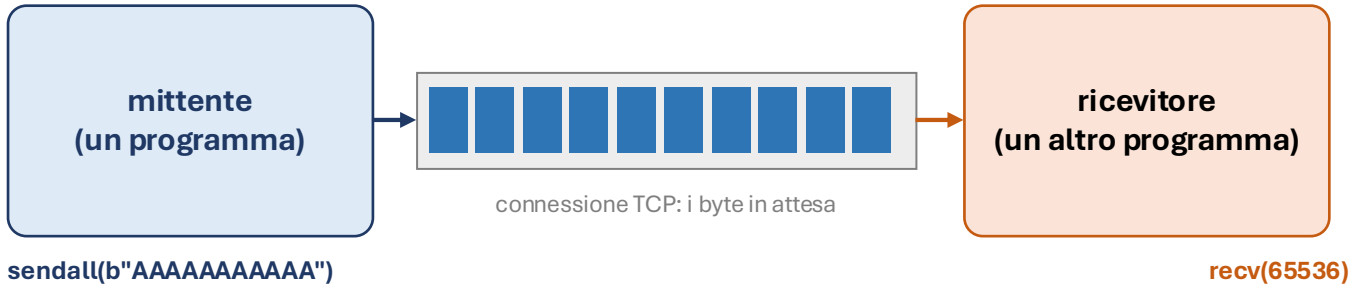
# poi: sendall da una parte, recv dall'altra
```

accept() restituisce un **socket nuovo**, conn, dedicato a quel client: i byte si leggono da conn, non da server. server serve solo ad accettare; alla fine si chiudono tutti e due.

Documentazione di Python, modulo socket (create_server, getsockname, accept, create_connection).

Socket TCP: sendall e recv

Un socket è il punto da cui un programma manda e riceve byte su una connessione. Con TCP, i due programmi aprono una connessione e poi si scambiano byte come in un tubo: quello che entra da un lato esce dall'altro, nello stesso ordine.



funzione	che cosa fa
sendall(dati)	consegna tutti i byte di dati alla connessione
recv(n)	restituisce i byte arrivati fino a quel momento, al massimo n; se non c'è niente aspetta
recv vuota: b""	il mittente ha chiuso la connessione

```
import socket

# il mittente
s = socket.create_connection(("127.0.0.1", porta))
for k in range(10):
    s.sendall(b"A" * 11)   # 10 volte 11 byte
s.close()

# il ricevitore, su una connessione accettata
while True:
    dati = conn.recv(65536)  # fino a 65536 byte
    if not dati:             # b"": ha chiuso
        break
    print(len(dati))
```

TCP manda una fila di byte e non tiene traccia di dove finisce un messaggio e comincia il successivo. Se il ricevitore legge quando sono già arrivati più messaggi, una sola recv gli restituisce tutti i byte attaccati, e non sa più separarli.

Documentazione di Python, modulo socket (create_connection, sendall, recv).

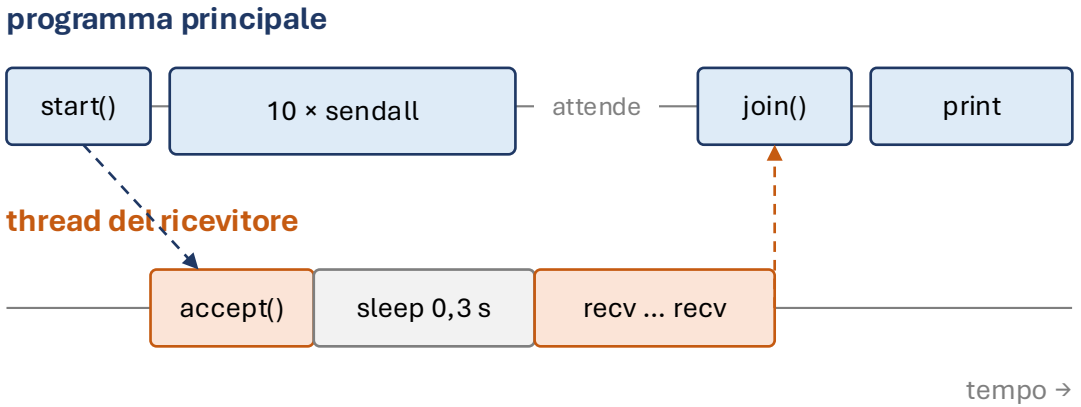
Python: thread

Un **thread** è un secondo filo di esecuzione nello stesso programma. Serve quando una parte deve **aspettare** (accept, recv) mentre l'altra lavora: qui il ricevitore aspetta, il programma principale manda.

```
import threading

def ricevitore(server):
    ...                    # accept, sleep, recv

t = threading.Thread(target=ricevitore,
                     args=(server,))
t.start()     # parte; il programma va avanti subito
...           # intanto il mittente manda
t.join()      # aspetta che il ricevitore finisca
print(ricevuti)   # solo ora la lista è completa
```



join() è il punto in cui il programma principale si ferma ad aspettare: riparte solo quando il ricevitore ha finito di leggere (freccia arancio). Solo da lì print vede tutti i byte ricevuti.

join() ferma il programma principale finché il ricevitore non ha finito. Senza join, il print partirebbe mentre il ricevitore sta ancora dormendo: la lista ricevuti sarebbe **vuota o incompleta**.

Documentazione di Python, modulo threading (Thread, start, join).

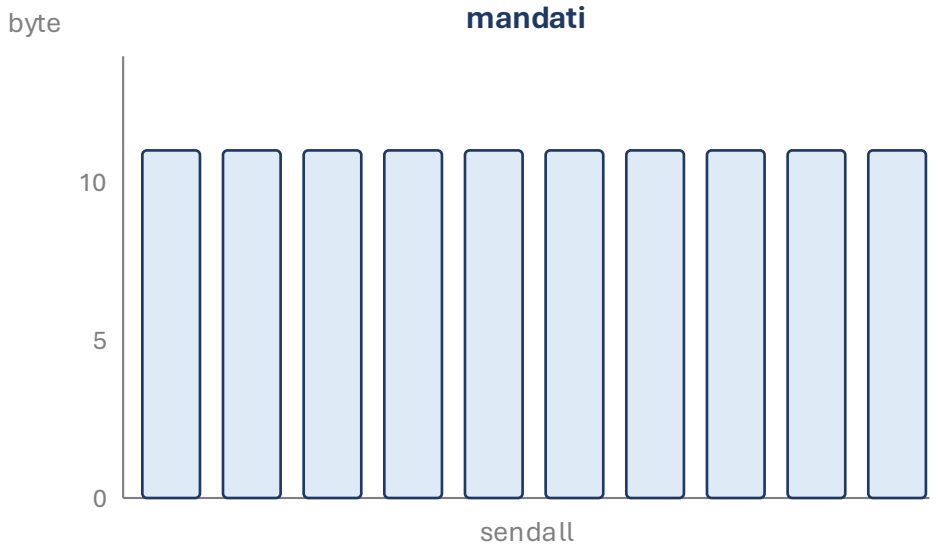
Python: matplotlib in breve

Un grafico si fa in tre mosse: si crea la **figura** con i suoi assi, si **disegna** (barre, linee, punti), si **scrive** che cosa dicono gli assi. Le stesse poche righe servono per tutti i grafici del laboratorio.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 3))   # 1. figura e assi
ax.bar([1, 2, 3], [11, 11, 11])          # 2. barre: x, altezze
# ax.plot(x, y, "o-")                    #    linee e punti
# ax.barh(nomi, valori)                   #    barre orizzontali
ax.set(xlabel="sendall", ylabel="byte",   # 3. che cosa dicono
       title="mandati")                   #    gli assi
plt.show()                               # mostra il grafico

fig, (a1, a2) = plt.subplots(1, 2)        # due grafici affiancati
```



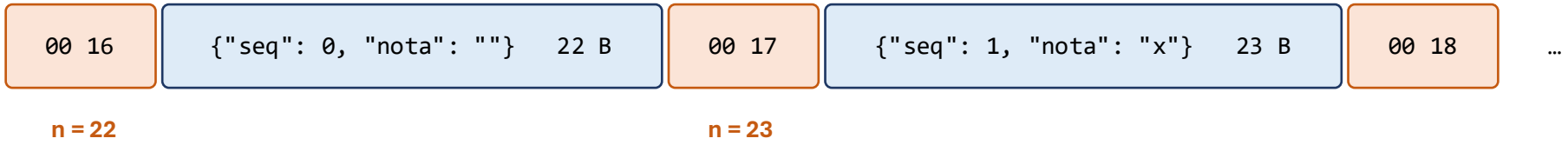
Un grafico senza titoli e unità sugli assi non si legge: ax.set(xlabel=..., ylabel=..., title=...) va sempre messo. Con valori che vanno da decine a migliaia, la scala logaritmica (set_xscale("log")) li fa stare tutti dentro.

Documentazione di matplotlib: pyplot.subplots, Axes.bar, Axes.plot, Axes.set.

La lunghezza davanti, byte per byte

Nella seconda cella ogni messaggio JSON parte preceduto da **2 byte con la sua lunghezza**. Il ricevitore legge prima quei 2 byte, poi **esattamente** quanti byte dicono: così ritrova i messaggi anche se le recv li tagliano ovunque.

il flusso di byte che parte dal mittente



il ricevitore, in ciclo

1 · leggi 2 byte

```
testa = leggi_esattamente(conn, 2)
n = struct.unpack("!H", testa)[0]
```

2 · leggi n byte

```
msg = leggi_esattamente(conn, n)
→ un messaggio intero
```

3 · ricomincia

```
finché recv restituisce b"":
il mittente ha chiuso
```

leggi_esattamente(conn, n) richiama recv finché non ha n byte: una sola recv può restituire meno di quanto si chiede. È lo stesso principio della Remaining Length di MQTT.

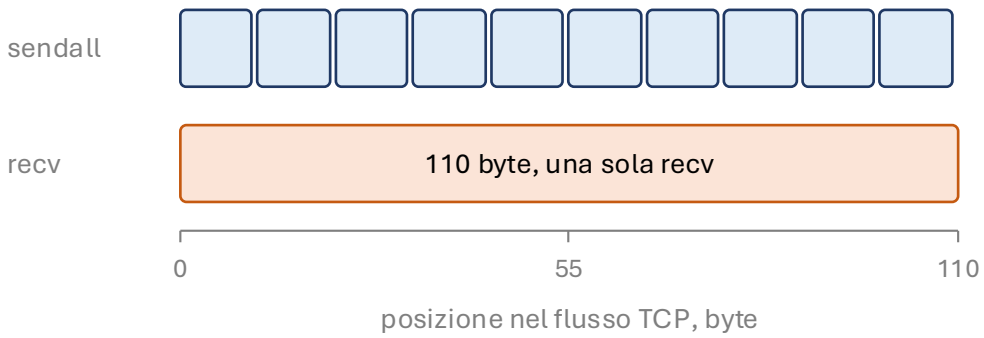
*Documentazione di Python: struct.pack("!H", n), socket.recv; messaggi della cella: json.dumps({"seq": k, "nota": "x" * k}).*

Parte 1 — TCP consegna byte

Due celle. Nella prima, un programma manda **10 messaggi da 11 byte** a un ricevitore che si mette a leggere con 0,3 s di ritardo. Nella seconda, davanti a ogni messaggio scrive **2 byte con la sua lunghezza**.

ricevitore	su un secondo thread: accetta la connessione, aspetta 0,3 s, poi legge finché il mittente non chiude
mittente	si collega e chiama sendall 10 volte, 11 byte ogni volta; poi chiude
risultato	la lista ricevuti: quanti byte ha restituito ogni recv

la figura della prima cella



ogni blocco è una chiamata, largo quanti byte ha passato: le linee bianche in alto sono i confini dei messaggi, in basso non ci sono più

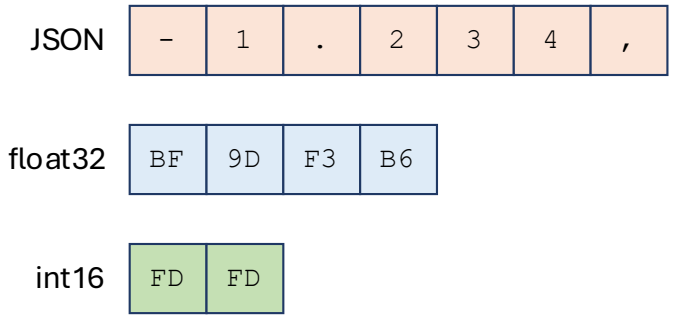
Nella prima cella i 110 byte arrivano in **una sola recv**: i confini fra i messaggi sono spariti. Nella seconda il ricevitore ritrova **tutti i messaggi, identici**, perché legge prima la lunghezza e poi esattamente quei byte.

Teoria: Cloud e MQTT, «Scope, transport and versions», «Remaining Length encoding».

Teoria: il formato dei dati

TCP e MQTT trasportano byte e non sanno che cosa significano: chi manda e chi riceve si accordano prima su un formato. Lo stesso numero può costare da 2 a 7 byte.

un campione: a_z = −1,234 m/s²

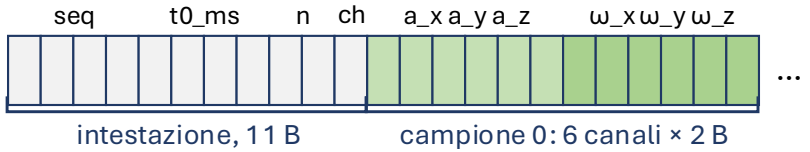


testo: 1 B per carattere;
la lunghezza dipende dal valore

IEEE 754: 4 B,
circa 7 cifre significative

round(−1,234 / q) = −515
q = 78,45 / 32 767 = 0,0024 m/s²

una finestra in int16 (pack_window)



Il fondo scala non viaggia: lo conoscono entrambi i lati, come il formato. Oltre il fondo scala il valore satura a ±32 767.
Le 36 feature: 36 × 4 B in float32, qualunque sia la lunghezza della finestra.

In int16 il valore torna −515 · q = −1,2331: l'errore è al massimo mezzo passo, 0,0012 m/s². Il JSON si legge a occhio ma rip ete nomi e cifre di ogni valore; float32 e int16 occupano sempre lo stesso numero di byte.

Teoria: Cloud e MQTT, «PUBLISH packet», «Three candidate payloads».

Python: numpy, la finestra

Una finestra è una tabella di numeri: **50 righe**, i campioni, per **6 colonne**, i canali. numpy la tiene in un array, e X.shape vale (50, 6).

riga

	x_acc	y_acc	z_acc	x_gyro	y_gyro	z_gyro
0	0.01	0.67	9.84	0.33	0.18	−0.01
1	0.18	0.86	10.70	0.37	0.16	0.05
2	0.41	1.09	11.35	0.40	0.13	0.12
…	…	…	…	…	…	…
49	−0.80	−0.97	9.51	−0.18	0.09	−0.40

X: 50 campioni (righe) × 6 canali (colonne)

```
t0_ms, X = synthetic_window()
X.shape        # (50, 6)
X[0]           # riga 0: un campione
X[:, 2]        # colonna 2: z_acc,
               # 50 numeri
X[:, 0:3]      # colonne 0, 1, 2:
               # l'accelerometro
```

bordo blu: X[0]

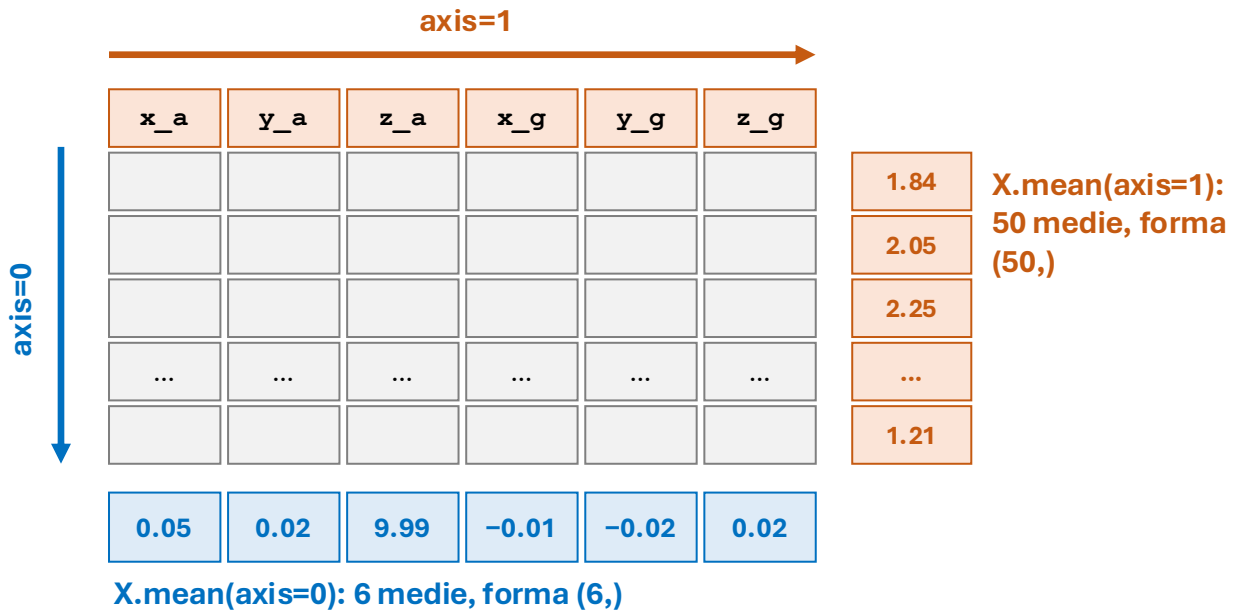
bordo arancio: X[:, 2]

Fra parentesi quadre prima le **righe**, poi le **colonne**. I due punti **:** vogliono dire «tutte»; **0:3** vuol dire 0, 1 e 2, con il 3 escluso.

Documentazione di numpy: ndarray.shape, indexing, mean, std, concatenate, astype, tobytes, frombuffer. Finestra: synthetic_window() del notebook.

Python: numpy, il parametro axis

mean e std chiedono **lungo quale direzione** calcolare. La regola: la direzione indicata da axis è quella che **sparisce** nel risultato.



chiamata	che cosa sparisce	forma
`X.mean(axis=0)`	i 50 campioni	(6,)
`X.mean(axis=1)`	i 6 canali	(50,)
`X.mean()`	tutto	un numero: 1.68
`X.std(axis=0)`	i 50 campioni	(6,)

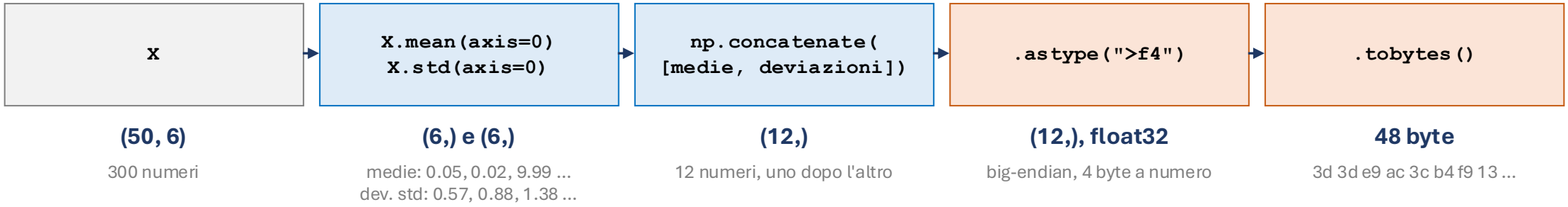
per ricordarlo: (50, 6) con axis=0 → (6,); con axis=1 → (50,)

Per il riassunto della finestra serve **axis=0**: una media e una deviazione standard per ogni canale, 6 + 6 numeri. axis=1 mescolerebbe m/s^2 e rad/s.

Documentazione di numpy: ndarray.shape, indexing, mean, std, concatenate, astype, tobytes, frombuffer. Finestra: synthetic_window() del notebook.

Python: numpy, dal riassunto ai byte

Dalle 300 misure della finestra a **48 byte**: media e deviazione standard di ogni canale, in un solo array, in float32, in byte.



lo stesso schema con massimo e minimo (il riassunto lo scrivete voi)

```
massimi = X.max(axis=0)           # (6,)
minimi = X.min(axis=0)            # (6,)
b = np.concatenate([massimi, minimi]).astype(">f4").tobytes()
len(b)                            # 48
```

std, la deviazione standard: quanto i valori si allontanano dalla media. Da fermi è vicina a 0, camminando cresce.

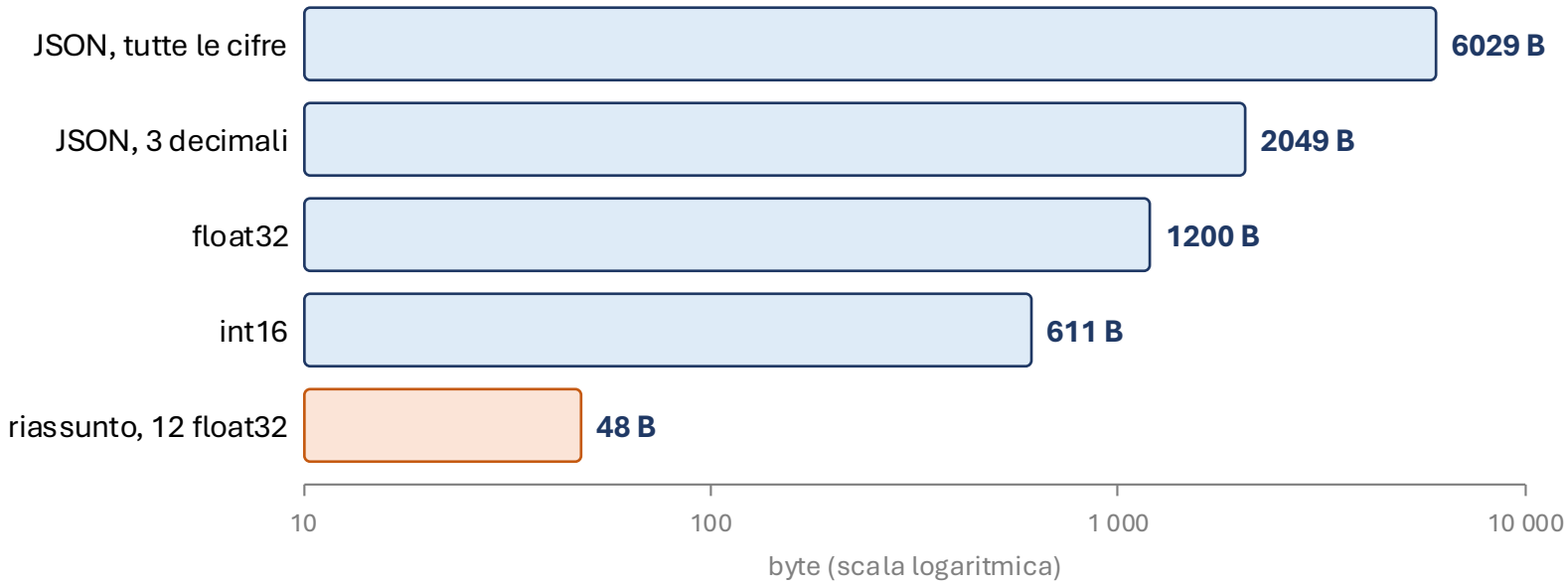
Il formato **">f4"** dice due cose: **>** big-endian, come «!» in struct; **f4** float32, 4 byte a numero. Chi riceve rilegge i 12 numeri con np.frombuffer(b, dtype=">f4").

Documentazione di numpy: ndarray.shape, indexing, mean, std, concatenate, astype, tobytes, frombuffer. Finestra: synthetic_window() del notebook.

Parte 2 — Il formato del payload

La stessa finestra di dati, **50 campioni × 6 canali** (1 s a 50 Hz), scritta in quattro formati diversi, più un riassunto di 12 numeri. Si contano i byte di ciascuno.

byte per finestra, scala logaritmica



poi lo «Scrivi tu»: invece dei dati, si manda solo la decisione, in 6 byte

Il JSON con tutte le cifre pesa **più di 100 volte** il riassunto. Il formato non lo decide la rete: è una **scelta di progetto**, fra leggibilità, precisione e byte.

Teoria: Cloud e MQTT, «PUBLISH packet», «Three candidate payloads». Byte calcolati con la libreria del laboratorio sulla finestra sintetica del notebook.

Una decisione in 6 byte

Invece della finestra, il nodo può mandare solo la sua conclusione, la **decisione**: un istante, una classe da 0 a 4 e una confidenza fra 0 e 1. Scrivete due funzioni che la trasformano in **6 byte** e viceversa.

```
def pack_decision(t_ms, classe, confidenza):
    # 6 byte, big-endian: struct.pack("!IBB", ...)
    # t_ms: 4 byte; classe: 1 byte;
    # confidenza in percento, da 0 a 100: 1 byte

def unpack_decision(b):
    # il contrario: struct.unpack("!IBB", b)
```

il controllo: questi 6 byte



pack_decision(123456, 2, 0.934) deve dare esattamente questi byte

poi si misura

i byte in rete di una decisione in 6 byte, della stessa decisione in JSON e della finestra int16, a QoS 1: quanti sono intestazione?

Il controllo nella cella successiva dice se i byte sono giusti; se non avete finito, il notebook va avanti lo stesso. Poi si confronta il costo in rete della decisione con quello della finestra.

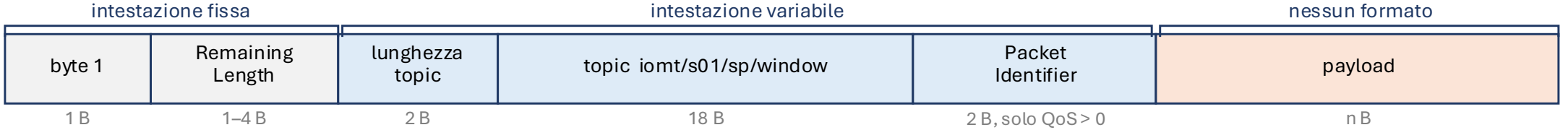
Teoria: Cloud e MQTT, «Three candidate payloads». Il formato "!IBB" è spiegato nella slide «Python: struct, numeri in byte».

Teoria: il pacchetto PUBLISH

Ogni pacchetto MQTT comincia con 1 byte che dice il tipo, seguito dalla Remaining Length: quanti byte vengono dopo. Il PUBLISH aggiunge il topic, scritto per intero, e, con QoS 1 o 2, un numero identificativo di 2 byte.



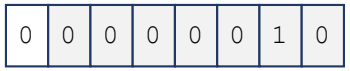
il byte 1, bit per bit. Bit 7–4: tipo 3 = PUBLISH; bit 3–0: i flag (DUP, QoS su due bit, RETAIN)



Remaining Length: 7 bit di valore per byte, il bit 7 dice «continua». Esempio: 321 = 2 · 128 + 65



0xC1: 65, continua



0x02: 2, ultimo

Remaining Length = 2 + len(topic) + 2 · [QoS > 0] + n

Le conferme (PUBACK, PUBREC, PUBREL, PUBCOMP) sono 4 byte ciascuna. Esempio: QoS 0, topic di 18 caratteri, 10 byte di dati: 1 + 1 + 2 + 18 + 10 = 32 byte, di cui 20 solo per il topic.

Teoria: Cloud e MQTT, «Fixed header», «Remaining Length encoding», «PUBLISH packet».

Parte 3 — MQTT contro TCP

Quattro messaggi, dalla decisione da 6 byte alla finestra in JSON. Per ciascuno si calcolano i byte con le regole del pacchetto PUBLISH, a QoS 0, 1 e 2, e con TCP e la lunghezza davanti. Le conferme sono comprese.

	dati	TCP + lunghezza	MQTT QoS 0	MQTT QoS 1	MQTT QoS 2
decisione, 6 byte	6	8	28	34	42
riassunto	48	50	70	76	84
finestra int16	611	613	634	640	648
finestra JSON	2 049	2 051	2 072	2 078	2 086

byte per messaggio, conferme comprese, calcolati con le regole del pacchetto PUBLISH

MQTT aggiunge circa **20 byte a messaggio**, quasi tutti per il topic; QoS 1 e 2 aggiungono pacchetti di conferma da 4 byte. Con i 6 byte della decisione l'intestazione è da quasi 4 a 6 volte i dati (22 byte in più a QoS 0, 36 a QoS 2); con la finestra intera conta poco.

Teoria: Cloud e MQTT, «PUBLISH packet», «Header and payload size», «QoS 0, 1 and 2 exchanges». Topic di 18 caratteri.